UNIVERSIDAD AUTÓNOMA DE ZACATECAS

"Ingeniería de Software"

UNIDAD ACADÉMICA DE INGENIERÍA ELÉCTRICA

Introducción a la Programación

Proyecto Final: Sistema SIGAP-UAIE con Tkinter

CAFETERÍA PAD

Nombres: Diana Patricia Morales Hernandez

Daniela Azeneth Quiñones Medellin

Alexis Armando Reyes Sanchez (LÍDER DEL EQUIPO)

Nombre Maestro: Santiago Esparza Guerrero

Grado y grupo: 2do A

Fecha de entrega: 19/05/2026

FOTOS DEL EQUIPO Y MATRÍCULAS:

43427105 ***42202736*** ***42204403***

INDICE

1. Indice
2. Descripción del problema
   2.1 ¿Qué hace el programa?
   2.2 Finalidad del programa
   2.3 ¿Que busca resolver?
   2.4 Requerimientos Funcionales
   2.5 Requerimientos no Funcionales
3.Problema a resolver
   3.1 Descripción del problema
   3.2 Entradas, procesos y salidas
4. Herramientas Utilizadas
   4.1 Herramientas usadas
5. Programa (Código fuente)
   5.1 Primer código (Hecho por nosotros)
   5.2 Segundo código (Decorado con ayuda de la ia)
6. Pruebas del sistema
7. Evidencias Visuales
   7.1 Captura completa
   7.2 Pantalla principal
   7.3 Pantalla orden producto
   7.4 Pantalla para agg nuevos productos
   7.5 Pantalla carrito
8. Conclusiones

2. Descripción del problema

2.1 ¿Qué hace el programa?

Nuestro programa llamado “CAFETERÍA PAD” es un sistema de gestión para una cafetería, lo desarrollamos en Python usando una interfaz gráfica basada en tkinter.

La funcionalidad de nuestro programa es permitir al usuario visualizar los productos (ya ordenados por categorías como pasteles, frappés, galletas) para agregarlos en una orden de compra y poder calcular automáticamente el total a pagar, además de también permitir poder registrar nuevos productos en el menú de la cafetería.

2.2 Finalidad del programa

Facilitar la gestión básica de ventas y pedidos en una cafetería, por ello nuestro programa ayuda a organizar los productos de manera más rápida, además de poder agregar pedidos/productos fácilmente, logrando que el proceso de atención al cliente sea mucho más fácil y efectivo.

2.3 ¿Que busca resolver?

La mala organización y los errores de precio, ya que en muchos pequeños negocios, siguen usando la forma tradicional y suelen tener confusiones en los productos y en el precio total a pagar.

2.4 Requerimientos Funcionales

- El sistema debe mostrar los productos disponibles de la cafetería.
- El sistema debe organizar los productos por categorías.
- El usuario debe poder seleccionar una categoría del menú.
- El sistema debe mostrar únicamente los productos de la categoría seleccionada.
- El usuario debe poder agregar productos al carrito de compra.
- El sistema debe calcular automáticamente el total a pagar.
- El sistema debe mostrar los productos agregados en la orden del cliente.
- El usuario debe poder registrar nuevos productos en el sistema.
- El sistema debe validar que los campos de nombre y precio no estén vacíos.
- El sistema debe mostrar mensajes de error cuando los datos sean incorrectos.

2.5 Requerimientos no Funcionales

- El sistema debe tener una interfaz gráfica sencilla y fácil de usar.

- El programa debe contar con una presentación visual ordenada y clara.
- El programa debe responder rápidamente a las acciones del usuario.
- El sistema debe ser entendible para usuarios con conocimientos básicos de computación.
- El código debe estar organizado y comentado.

3.Problema a resolver

3.1 Descripción del problema

El problema que identificamos es la falta de organización sobre atención al cliente en una cafetería ya que las anotaciones son a mano lo que complica el tema de organización llevando a que la atención al cliente no sea el adecuado desembocando en una disminución de ingresos puesto que no existe una gráfica clara sobre el movimiento de activos y financiero, a continuación se describen algunos de las mejoras que podemos llevar acabo.

Pedidos: El software es para mejorar la administración de una cafetería ayudando a tener un a mejor uso de los activos de recursos humanos

Ingresos al contemplar el uso de un software ayudamos a la cafetería a tener un mejor flujo de ingreso con una gráfica clara y específica:

- Egresos
- Compras
- Elaboracion de Producto Final

Esto implica que haya errores en pedidos, pagos y registro de productos en disponibilidad base a estos punto se creó un sistema software con una interfaz gráfica que convenga para saber productos en existencia,se lleve un registro de las cosas compradas y permita mejor atencion al clientes así mejorando todo para la cafetería haciéndola mejor

3.2 Entradas, procesos y salidas

- Entradas:

  -Texto con nombre del artículo.

  -Categoría seleccionada.

  -Texto con el precio.

- Botón de agregar producto "Agregar Producto".

- Procesos

  –agregar producto

  -total de sumatoria de productos

  -reseteo de campos

  -guardar producto

- Salidas

  -al agregar producto y llenar todo lo pedido se limpian los cuadro de nombre y producto

  - aviso por si no se llena un cuadro

  -aviso de que se agrego el producto a nombre de alguien

  -total marcado

4. Herramientas Utilizadas

4.1 Herramientas usadas

Estas son las herramientas usadas en los códigos, nosotros creamos el primer código sin nada de ia, para el segundo código tomamos la ayuda de chat gpt para la decoración del programa, haciéndolo ver más profesional y bonito.

Primer codigo:

- Python → se utilizó para desarrollar el funcionamiento del sistema.
- Tkinter → se utilizó para crear la interfaz gráfica del programa.
- **ttk** → se utilizó para crear menús desplegables y componentes más modernos.
- **messagebox** → se utilizó para mostrar mensajes de error, aviso y confirmación.

- **Listas y diccionarios de Python** → se utilizan para almacenar los productos y el carrito de compras.
- **Funciones en Python** → se utilizaron para organizar las acciones del sistema, como mostrar productos y calcular el total.

Segundo codigo:

- **Python** → lenguaje principal utilizado para desarrollar el sistema.
- **Tkinter** → librería usada para crear la interfaz gráfica del programa.
- **GUI (Interfaz Gráfica de Usuario)** → el sistema funciona mediante ventanas, botones, listas y cuadros de texto
- **Listas y diccionarios de Python** → estructuras utilizadas para almacenar los productos, categorías y carrito de compras.
- **Funciones en Python** → utilizadas para organizar las acciones del sistema, como mostrar productos y agregar al carrito.
- **Eventos y comandos** → usados para que los botones ejecuten acciones cuando el usuario hace clic.
- **Chat GPT** → se utilizó como apoyo para decorar la interfaz, mejorar el diseño visual y resolver dudas durante el desarrollo del programa.

5. Programa (Código fuente)

5.1 Primer código (Hecho por nosotros)

```
import tkinter as tk

from tkinter import ttk, messagebox  #Importamos las librerias
necesarias



#Productos de la cafetería

productos = [

    # Pasteles
```

```
    {"nombre": "Pastel de Chocolate", "tipo": "Pasteles", "precio": 85},
    {"nombre": "Cheesecake de Fresa", "tipo": "Pasteles", "precio": 95},
    {"nombre": "Pastel Red Velvet", "tipo": "Pasteles", "precio": 90},
    {"nombre": "Pastel de Vainilla", "tipo": "Pasteles", "precio": 80},
    {"nombre": "Pay de Limón", "tipo": "Pasteles", "precio": 75},

    # Galletas
    {"nombre": "Galleta Chispas de Chocolate", "tipo": "Galletas", "precio": 30},
    {"nombre": "Galleta Red Velvet", "tipo": "Galletas", "precio": 35},
    {"nombre": "Galleta de Avena", "tipo": "Galletas", "precio": 28},
    {"nombre": "Galleta Oreo", "tipo": "Galletas", "precio": 32},
    {"nombre": "Galleta de Mantequilla", "tipo": "Galletas", "precio": 25},

    # Bebidas
    {"nombre": "Café Latte", "tipo": "Bebidas", "precio": 65},
    {"nombre": "Capuchino", "tipo": "Bebidas", "precio": 70},
    {"nombre": "Chocolate Caliente", "tipo": "Bebidas", "precio": 60},
    {"nombre": "Té Chai", "tipo": "Bebidas", "precio": 58},
    {"nombre": "Matcha Latte", "tipo": "Bebidas", "precio": 75},

    # Frappés
```

```
    {"nombre": "Frappé Mocha", "tipo": "Frappés", "precio": 85},
    {"nombre": "Frappé Oreo", "tipo": "Frappés", "precio": 90},
    {"nombre": "Frappé Caramelo", "tipo": "Frappés", "precio": 88},
    {"nombre": "Frappé Fresa", "tipo": "Frappés", "precio": 80},
    {"nombre": "Frappé Vainilla", "tipo": "Frappés", "precio": 82}
]


carrito = [] #lista para saber que comprara el cliente
tipos = ["Pasteles", "Galletas", "Bebidas", "Frappés"]



def mostrar_por_categoria(): #Limpia los productos y solo muestra los de la categoria seleccionada
    categoria = combo_categoria.get()
    listbox_productos.delete(0, tk.END)
    for producto in productos:
        if producto["tipo"] == categoria:
            texto = f"{producto['nombre']} - ${producto['precio']}"
            listbox_productos.insert(tk.END, texto)


def agregar_al_carrito():
    seleccion = listbox_productos.curselection() #Selecciona por medio del cursor
    if not seleccion:
        messagebox.showwarning("Aviso", "Seleccione un producto.") #Mensajes para cuando haya errores
```

```
        return


    texto = listbox_productos.get(seleccion[0])
    nombre = texto.split(" - $")[0]


    for producto in productos:
        if producto["nombre"] == nombre:
            carrito.append(producto) #abre la lista de productos
            listbox_carrito.insert(tk.END, texto)
            actualizar_total() #va actualizando el precio
            break


def actualizar_total():
    total = sum(p["precio"] for p in carrito)
    label_total.config(text=f"Total: ${total}") #funcion pequeña para
ir sumando el total


def agregar_producto_manual(): #es la parte donde se agregaran los
productos nuevos
    nombre = entry_nombre.get().strip()
    tipo = combo_tipo_nuevo.get()
    precio_str = entry_precio.get().strip()


    if not nombre or not precio_str:
        messagebox.showerror("Error", "Nombre y precio son
obligatorios.") #Mensajes por si no se completan espacios
```

```
        return


    try:
        precio = float(precio_str)
    except ValueError:
        messagebox.showerror("Error", "El precio debe ser un número.")
        return


    productos.append({"nombre": nombre, "tipo": tipo, "precio":
precio}) #Se agrega a las listas de productos
    messagebox.showinfo("Éxito", f"Producto '{nombre}' agregado.")
    entry_nombre.delete(0, tk.END)
    entry_precio.delete(0, tk.END)


    if combo_categoria.get() == tipo:
        mostrar_por_categoria()


# Toda la parte de la interfaz y titulo del programa


root = tk.Tk()
root.title("Cafetería PAD")


#Ver productos por la categoría
frame_categoria = tk.LabelFrame(root, text="Ver Productos por
Categoría")
frame_categoria.pack(padx=10, pady=5, fill="x")
```

```
combo_categoria = ttk.Combobox(frame_categoria, values=tipos,
state="readonly")

combo_categoria.pack(side="left", padx=5)

combo_categoria.set(tipos[0])


tk.Button(frame_categoria, text="Mostrar",
command=mostrar_por_categoria).pack(side="left")


# Poder ver la lista de productos

listbox_productos = tk.Listbox(root, height=10)

listbox_productos.pack(padx=10, pady=5, fill="both", expand=True)


tk.Button(root, text="Agregar al Carrito",
command=agregar_al_carrito).pack(pady=5)


# Para el carrito y total

frame_carrito = tk.LabelFrame(root, text="Carrito de Compras")

frame_carrito.pack(padx=10, pady=5, fill="x")


listbox_carrito = tk.Listbox(frame_carrito, height=5)

listbox_carrito.pack(side="left", fill="both", expand=True)


label_total = tk.Label(frame_carrito, text="Total: $0")

label_total.pack(side="right", padx=10)
```

```
#  Para poder agrerar productos nuevos a la lista, con su nombre, tipo
y precio

frame_nuevo = tk.LabelFrame(root, text="Agregar Producto Manualmente")

frame_nuevo.pack(padx=10, pady=10, fill="x")



tk.Label(frame_nuevo, text="Nombre:").grid(row=0, column=0)

entry_nombre = tk.Entry(frame_nuevo)

entry_nombre.grid(row=0, column=1)



tk.Label(frame_nuevo, text="Tipo:").grid(row=0, column=2)

combo_tipo_nuevo = ttk.Combobox(frame_nuevo, values=tipos,
state="readonly")

combo_tipo_nuevo.grid(row=0, column=3)

combo_tipo_nuevo.set(tipos[0])



tk.Label(frame_nuevo, text="Precio:").grid(row=0, column=4)

entry_precio = tk.Entry(frame_nuevo)

entry_precio.grid(row=0, column=5)



tk.Button(frame_nuevo, text="Agregar Producto",
command=agregar_producto_manual).grid(row=0, column=6, padx=5)



# Ejecutar el programa

root.mainloop()
```

5.2 Segundo código (Decorado con ayuda de la ia)

```
import tkinter as tk

from tkinter import ttk, messagebox



# productos de la cafetería



productos = [



    # Pasteles
```

```
    {"nombre": "Pastel de Chocolate", "tipo": "Pasteles", "precio": 85},
    {"nombre": "Cheesecake de Fresa", "tipo": "Pasteles", "precio": 95},
    {"nombre": "Pastel Red Velvet", "tipo": "Pasteles", "precio": 90},
    {"nombre": "Pastel de Vainilla", "tipo": "Pasteles", "precio": 80},
    {"nombre": "Pay de Limón", "tipo": "Pasteles", "precio": 75},

    # Galletas
    {"nombre": "Galleta Chispas de Chocolate", "tipo": "Galletas", "precio": 30},
    {"nombre": "Galleta Red Velvet", "tipo": "Galletas", "precio": 35},
    {"nombre": "Galleta de Avena", "tipo": "Galletas", "precio": 28},
    {"nombre": "Galleta Oreo", "tipo": "Galletas", "precio": 32},
    {"nombre": "Galleta de Mantequilla", "tipo": "Galletas", "precio": 25},

    # Bebidas
    {"nombre": "Café Latte", "tipo": "Bebidas", "precio": 65},
    {"nombre": "Capuchino", "tipo": "Bebidas", "precio": 70},
    {"nombre": "Chocolate Caliente", "tipo": "Bebidas", "precio": 60},
    {"nombre": "Té Chai", "tipo": "Bebidas", "precio": 58},
    {"nombre": "Matcha Latte", "tipo": "Bebidas", "precio": 75},

    # Frappés
```

```
    {"nombre": "Frappé Mocha", "tipo": "Frappés", "precio": 85},
    {"nombre": "Frappé Oreo", "tipo": "Frappés", "precio": 90},
    {"nombre": "Frappé Caramelo", "tipo": "Frappés", "precio": 88},
    {"nombre": "Frappé Fresa", "tipo": "Frappés", "precio": 80},
    {"nombre": "Frappé Vainilla", "tipo": "Frappés", "precio": 82}
]


# Lista del carrito
carrito = []


# Categorías disponibles
tipos = ["Pasteles", "Galletas", "Bebidas", "Frappés"]




# mostrar productos por categoría
def mostrar_por_categoria():


    categoria = combo_categoria.get()


    listbox_productos.delete(0, tk.END)


    for producto in productos:


        if producto["tipo"] == categoria:
```

```
            texto = f"{producto['nombre']} - ${producto['precio']}"


            listbox_productos.insert(tk.END, texto)



# agg carrito


def agregar_al_carrito():


    seleccion = listbox_productos.curselection()


    if not seleccion:


        messagebox.showwarning(

            "Aviso",

            "Seleccione un producto del menú."

        )


        return


    texto = listbox_productos.get(seleccion[0])


    nombre = texto.split(" - $")[0]


    for producto in productos:
```

```
        if producto["nombre"] == nombre:

            carrito.append(producto)

            listbox_carrito.insert(tk.END, texto)

            actualizar_total()

            break



# precio total a pagar
def actualizar_total():

    total = sum(p["precio"] for p in carrito)

    label_total.config(text=f"Total a pagar: ${total}")



# poder agg productos nuevos al menú

def agregar_producto_manual():

    nombre = entry_nombre.get().strip()
```

```
tipo = combo_tipo_nuevo.get()


precio_str = entry_precio.get().strip()


if not nombre or not precio_str:


    messagebox.showerror(

        "Error",

        "El nombre y precio son obligatorios."

    )


    return


try:


    precio = float(precio_str)


except ValueError:


    messagebox.showerror(

        "Error",

        "El precio debe ser un número."

    )
```

```
        return


    productos.append({
        "nombre": nombre,
        "tipo": tipo,
        "precio": precio
    })


    messagebox.showinfo(
        "Producto agregado",
        f"'{nombre}' fue agregado al menú."
    )


    entry_nombre.delete(0, tk.END)


    entry_precio.delete(0, tk.END)


    if combo_categoria.get() == tipo:
        mostrar_por_categoria()



# interfaz


root = tk.Tk()
```

```
root.title("CAFETERÍA PAD")


root.geometry("800x750")


root.configure(bg="#f8ede3")




# Titulo del programa
titulo = tk.Label(
    root,
    text="☕ CAFETERÍA PAD ☕",
    font=("Arial", 22, "bold"),
    bg="#f8ede3",
    fg="#5c3d2e"
)


titulo.pack(pady=10)




# Las categorias
frame_categoria = tk.LabelFrame(
    root,
    text="Menú por Categoría",
    bg="#f8ede3",
    fg="#5c3d2e",
```

```
    font=("Arial", 10, "bold")
)

frame_categoria.pack(padx=10, pady=5, fill="x")

combo_categoria = ttk.Combobox(
    frame_categoria,
    values=tipos,
    state="readonly",
    font=("Arial", 10)
)

combo_categoria.pack(side="left", padx=5)

combo_categoria.set(tipos[0])

boton_mostrar = tk.Button(
    frame_categoria,
    text="Mostrar Productos",
    bg="#d0b49f",
    fg="black",
    font=("Arial", 10, "bold"),
    command=mostrar_por_categoria
)
```

```
boton_mostrar.pack(side="left", padx=5)



# Lista de productos disponibles
listbox_productos = tk.Listbox(
    root,
    height=14,
    font=("Arial", 11),
    bg="white"
)

listbox_productos.pack(
    padx=10,
    pady=10,
    fill="both",
    expand=True
)



# Botones para agregar al carrito

boton_agregar = tk.Button(
    root,
    text="Agregar a la Orden",
    bg="#c8b6a6",
```

```
    font=("Arial", 11, "bold"),
    command=agregar_al_carrito
)

boton_agregar.pack(pady=5)



# orden de compra y total a pagar

frame_carrito = tk.LabelFrame(
    root,
    text="🛒 Orden del Cliente",
    bg="#f8ede3",
    fg="#5c3d2e",
    font=("Arial", 10, "bold")
)

frame_carrito.pack(
    padx=10,
    pady=10,
    fill="x"
)

listbox_carrito = tk.Listbox(
    frame_carrito,
```

```
    height=6,
    font=("Arial", 10)
)

listbox_carrito.pack(
    side="left",
    fill="both",
    expand=True,
    padx=5,
    pady=5
)

label_total = tk.Label(
    frame_carrito,
    text="Total a pagar: $0",
    font=("Arial", 12, "bold"),
    bg="#f8ede3",
    fg="#5c3d2e"
)

label_total.pack(side="right", padx=15)

# Poder agregar productos
```

```
frame_nuevo = tk.LabelFrame(
    root,
    text="➕ Agregar Nuevo Producto",
    bg="#f8ede3",
    fg="#5c3d2e",
    font=("Arial", 10, "bold")
)

frame_nuevo.pack(
    padx=10,
    pady=10,
    fill="x"
)

# Nombres de productos
tk.Label(
    frame_nuevo,
    text="Nombre:",
    bg="#f8ede3",
    font=("Arial", 10)
).grid(row=0, column=0, padx=5, pady=5)

entry_nombre = tk.Entry(
    frame_nuevo,
    font=("Arial", 10)
```

```
)


entry_nombre.grid(row=0, column=1)



# Categorias
tk.Label(
    frame_nuevo,
    text="Categoría:",
    bg="#f8ede3",
    font=("Arial", 10)
).grid(row=0, column=2, padx=5)

combo_tipo_nuevo = ttk.Combobox(
    frame_nuevo,
    values=tipos,
    state="readonly",
    font=("Arial", 10)
)

combo_tipo_nuevo.grid(row=0, column=3)

combo_tipo_nuevo.set(tipos[0])
```

```
# Precios
tk.Label(
    frame_nuevo,
    text="Precio:",
    bg="#f8ede3",
    font=("Arial", 10)
).grid(row=0, column=4, padx=5)

entry_precio = tk.Entry(
    frame_nuevo,
    font=("Arial", 10)
)

entry_precio.grid(row=0, column=5)


# Botón agregar producto
tk.Button(
    frame_nuevo,
    text="Agregar Producto",
    bg="#d0b49f",
    font=("Arial", 10, "bold"),
    command=agregar_producto_manual
).grid(row=0, column=6, padx=10)
```

```
# Poder ejecutar el programa y mostrar los productos por categoría

mostrar_por_categoria()

root.mainloop()
```

6. Pruebas del sistema

Caso	Entrada (Acción del usuario)	Resultado esperado	Resultado obtenido
01	Seleccionar la categoría "Pasteles" en el combo y presionar el botón "Mostrar Productos".	La lista principal debe limpiarse y mostrar únicamente los 5 pasteles cargados por defecto con sus respectivos precios.	Desplegó correctamente solo los productos pertenecientes a la categoría seleccionada.
02	Dar clic en el botón "Agregar a la Orden" sin haber seleccionado	El sistema debe interrumpir el flujo y lanzar una alerta de tipo aviso que diga: "Seleccione un producto del menú."	Se mostró la ventana de aviso flotante y no se alteró el carrito.

	ningún elemento del menú.		
03	Seleccionar "Café Latte - $65" de la lista y presionar "Agregar a la Orden".	El producto debe reflejar visualmente en la lista de la orden del cliente y la etiqueta del total debe cambiar a $65.	El elemento se agregó al listbox del carrito y el total se actualizó al instante.
04	Agregar de forma consecutiva un "Cheesecake de Fresa" ($95) y un "Frappé Mocha" ($85).	El acumulador de la compra debe sumar los precios correctamente y mostrar en la esquina inferior: "Total a pagar: $180".	La suma se ejecutó sin errores; el total reflejó la cantidad exacta de $180.
05	Intentar registrar un nuevo producto dejando los campos de "Nombre" o "Precio" vacíos.	El programa debe detectar la ausencia de datos y mostrar un mensaje de error indicando que ambos campos son obligatorios.	Bloqueó el registro vacío y mostró la alerta de error esperada.

06	Registrar un nuevo producto escribiendo letras en el campo de precio (ej. Nombre: "Muffin", Precio: "gratis").	Al no poder convertirse a un valor numérico, el sistema debe cachar el error y mandar una alerta: "El precio debe ser un número."	El programa no se plasmó; detuvo el proceso y mandó el mensaje de error por tipo de dato inválido.
07	Introducir un producto válido (Nombre: "Afogatto", Categoría: "Bebidas", Precio: "72") y dar clic en "Agregar Producto".	Debe salir un mensaje de confirmación de éxito. Si se tiene seleccionada la categoría "Bebidas", el nuevo artículo debe aparecer al momento en la lista.	Se guardó el producto en la lista interna y se actualizó la interfaz de forma inmediata.

7. Evidencias Visuales

7.1 Captura completa



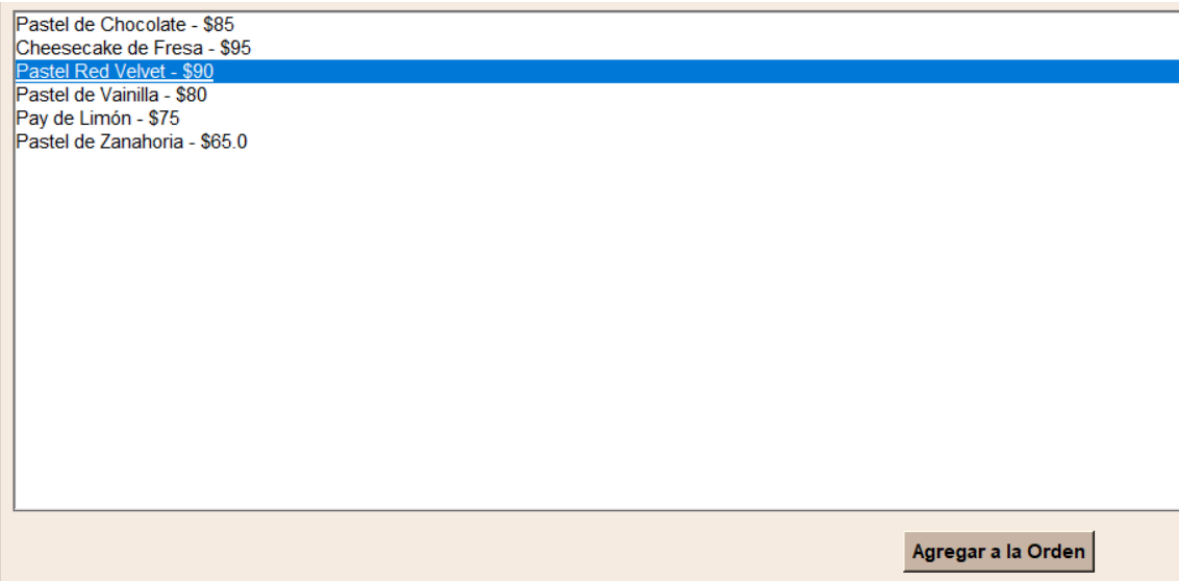
En esta captura observamos cómo luce el programa “Cafeteria PAD”

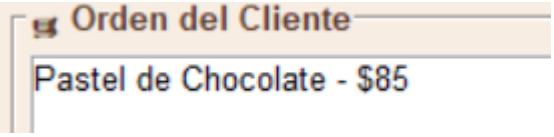
7.2 Pantalla principal



Al entrar nos encontramos con este menú y aquí podemos apreciar como se maneja este y botones “emergente” para poder interactuar con ellos.

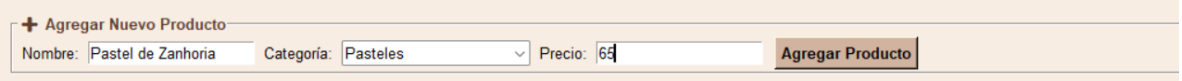
7.3 Pantalla orden producto





Aquí apreciamos donde irán todos los artículos que agregues desde el menú, solo es necesario seleccionar el producto y apretarle a “Agregar a la orden”.

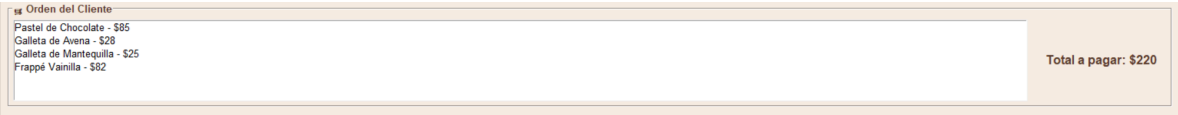
7.4 Pantalla para agg nuevos productos



Pastel de Chocolate - $85
Cheesecake de Fresa - $95
Pastel Red Velvet - $90
Pastel de Vainilla - $80
Pay de Limón - $75
Pastel de Zanahoria - $65.0

Aquí podemos agregar nuevos productos, ya sea pastel,galletas, bebidas o frappes, al igual que ponerles el precio, una vez agregadas esas tres cosas, le apretamos en el botón emergente y ya se podrá mostrar en el apartado de menú.

7.5 Pantalla carrito



Aquí se muestran todas los productos que llevas en el carrito, al igual que su total a pagar.

8. Conclusiones

Con todo esto podemos llegar a la conclusión de que el desarrollo de este proyecto representó una experiencia integral que va más allá de la simple escritura de código, el proceso del proyecto final nos permitió consolidar conocimientos clave en la estructuración de datos con Python y su integración real en interfaces gráficas mediante Tkinter, entendiendo cómo se conecta la lógica de un programa con la experiencia visual de un usuario.

Respecto a los desafíos superados, el mayor problema consistió en asegurar la estabilidad del sistema, especialmente en la validación de las entradas de datos manuales para evitar que caracteres inválidos o campos vacíos corrompieron la aplicación, resolver esto mediante el manejo de excepciones y flujos de control fue un gran problema que nos costó a nosotros pero se logró.

De esta manera este proyecto nos ayudo mucho por que nos contribuye a nuestro formamiento profesional al ponernos en un problema muy común entre la vida cotidiana, pero esto no lo hubiéramos podido hacer sin **los mandamientos de CHAGO UAZ.**